original order in which formulas are presented is actually important: it is perfectly possible to have a collection of axioms which produces a proof in some order but not in another. *Vampire* includes the option `--normalize` to prevent this effect. For *Prover9*, one can use a larger environment to automate the permutations of the input file and make several tries with time restrictions. In contrast, the order of the formulas does not affect *Mace4*'s response.

Moreover, the inclusion of additional formulas redundant with the premises may significantly speed up the proof search, if those formulas are not quickly derived by the ATP algorithm (e.g., evaluations mapping different variables to the same one are routinely included).

Finally, the use of demodulators can greatly simplify and speed up the proof search. In *Prover9*, the `assign(eq_defs, fold)` command allows to substitute any specified subexpression by a user-defined symbol, simplifying all further expressions generated by the ATP. For instance, a quick proof that E102744082 implies the injectivity of right multiplications is obtained by setting `eq_defs` and adding the formula `s(x) = x*x` (where `*` stands for $\diamond$) to the premises (together with a good weight limit, see Section 7.3.5). In *Vampire*, deactivating the `--function_definition_elimination` mode can serve a similar purpose.

7.3.5. *The weight-sos limit strategy.* In *Prover9*, each clause is assigned a weight depending on its length (and other customizable parameters), with newly generated clauses exceeding the `max_weight` limit being discarded. In addition, the size of the list of clauses awaiting processing (the SOS list) is controlled with the `sos_limit` parameter, with most newly generated clauses being discarded once the list is full. Consequently, the size and shape of the search space can be partially controlled through these parameters. Ideally, we would want to use the smallest values of `max_weight` and `sos_limit` that still guarantee a proof to be contained in the search space. Thus smaller values are preferable, provided they are not so low that the search is exhausted without finding a proof.

Since there are more parameters affecting the search, and different proofs may be reachable depending on the configuration, the system's behaviour with respect to `max_weight` and `sos_limit` is not straightforward, with several casuistics arising. When `sos_limit` is fixed, the proof-finding time tends to rise with `max_weight` until reaching a plateau, with the length of this increasing phase extending for higher `sos_limit` values (see Figure 13). For this reason we recommend lowering `sos_limit` from its default value of 20000 to substantially smaller values.

On the other hand, when `max_weight` is fixed, the smallest `sos_limit` values that still yield proofs typically produce a chaotic transient phase with higher proof-finding times, after which a better-defined relation between proof-finding time and `sos_limit` emerges, which tends to follow either a near-plateau pattern (Figure 14a)), or a monotonically increasing trend (Figure 14b)). We recommend avoiding an excessively low `sos_limit`, in order to prevent the onset of the transient phase.

Other relevant characteristics related to the complexity of the resulting proof (such as proof level, length, weight, etc.) may also vary in nontrivial ways depending on the chosen `sos_limit` value (see Figure 15).

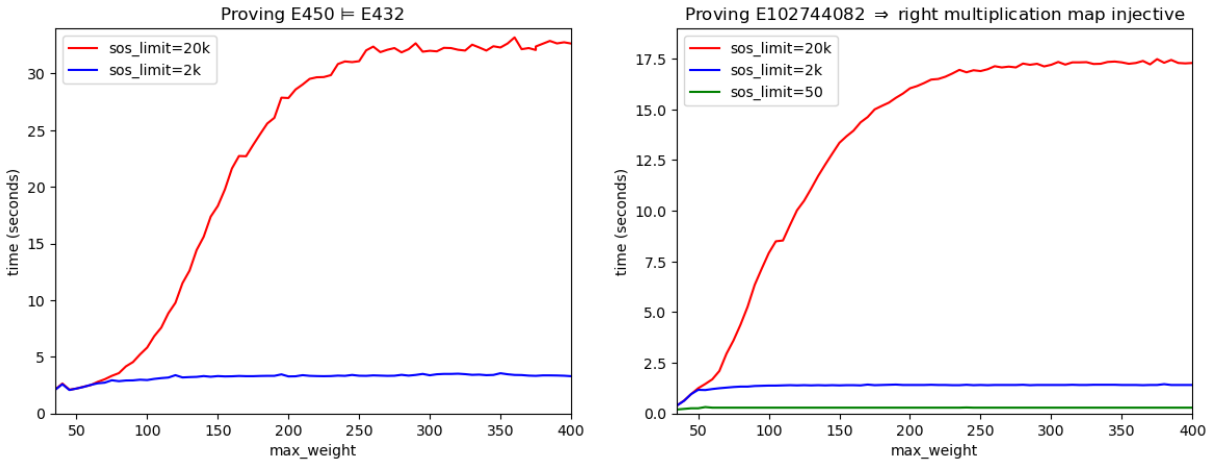

FIGURE 13. Proof-finding time as a function of `max_weight`, for several values of `sos_limit`.

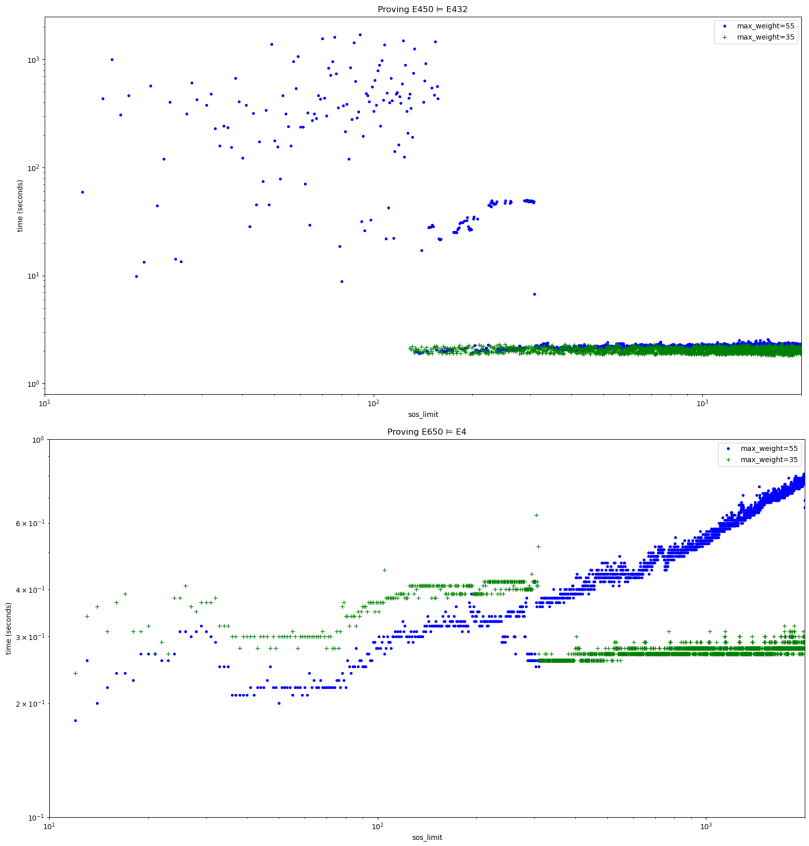

FIGURE 14. Proof-finding time as a function of `sos_limit`, for different values of `max_weight`. a) Stabilization as plateau, same for different weights. b) Different behaviours for different weights, with one trend monotonically increasing.

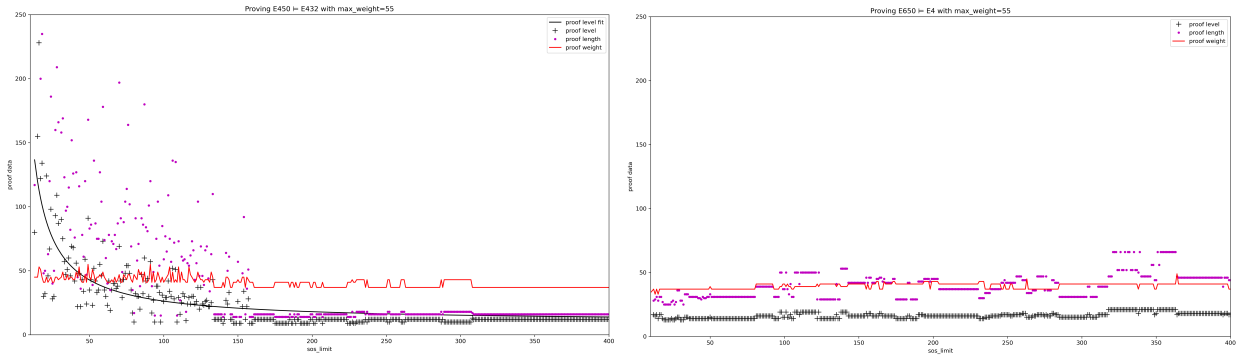

FIGURE 15. Proof-complexity indicators as a function of `sos_limit`.

Remarkably, *Prover9* can establish all positive implications between laws of order up to 4 using `max_weight` 55 and `sos_limit` 20000, with `max_weight` 25 being sufficient for almost every case. With default parameters (`max_weight` 100, `sos_limit` 20000), all consequences of each equation can be proven in at most 1 second per equation, with the exception of laws E450 and E650 (and their duals), which take somewhat longer[18].

Vampire does not allow the imposition of a user-defined maximum weight, although its default LRS algorithm already implements a dynamic weight limit. According to [32], *Vampire* 5.0 can prove all true implications from the ETP, and can prove 99.97% of them using less than 500 instructions per proof.

7.3.6. *Mace4.* The configuration of *Mace4* likewise has a substantial impact on its running time, with different parameter settings resulting in differences of orders of magnitude. In particular, model-computation time is greatly affected by the `selection_measure` and `skolems_last` parameters. The `selection_order` parameter is also relevant, but our empirical data indicate that `selection_order` 2 is by far the best general choice. It is outperformed by `selection_order` 0 only in a few cases, and even then the improvement in computing time is not that substantial. The effect of setting `skolems_last` can range from slightly detrimental to highly advantageous: as an example, a model of E272260 $\not\models$ E42323216 of size 7 is found in 0.0s with (2,4,Y), and in 4.0s with (2,4,N). Here and in the following we use notation (a,b,S) to mean the configuration `selection_order` a, `selection_measure` b, and `skolems_last` set (S=Y) or clear (S=N).

There is no universally optimal configuration (a,b,S): for each (potential) implication or theory, configurations may be ranked differently. However, in the ETP context, certain configurations have performed consistently better, roughly ordered as (2,3,Y), (2,1,Y), (2,4,Y), (2,3,N), (2,1,N), (2,4,N), (0,1,N), (0,1,Y), with the remaining configurations being largely avoidable. Problems involving several algebraic operations, or presenting some kind of symmetry (see Table 2 on weak central groupoids), tend to perform better with (2,4,N/Y). Problems may also be considerably sensitive to changes in formulas that appear minor, for example adding $f^n(x) = x$ with different values of n can result in different optimal configurations.

[18]For details, see `https://leanprover.zulipchat.com/#narrow/channel/458659-Equational/topic/Which.20implications.20are.20harder.20for.20ATPs/near/547089094`.